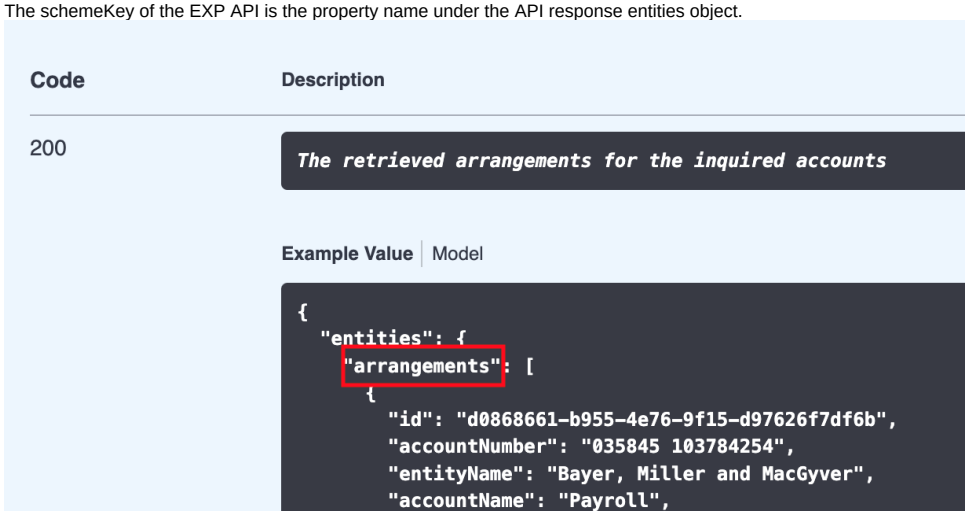
Build EXP with GitHub Copilot and VS Code Tasks

Prerequisites:

Get the following values from SD, EXP API Swagger, and Downstream API Swagger:

Field Name	Source	Example	
downstreamService	Mesh Connect	cap-prodsvcadm-proddealacctadm-instl-arrngmntsvcng-v1	
expApiOperationId	EXP API Contract (Swagger Editor)	getArrangements	
schemeKey	Soludtion Design	arrangements	The schemeKey of the EXP API is the property name under the API response entities object. 
downstreamApiName	Downstream API Contract (Swagger Editor)	getArrangements	
downstreamSwaggerUrl	Downstream API Contract (Swagger Editor)	https://schema.mesh.westpac.com.au/component/wdp-cap-prodsvcadm-proddealacctadm-instl-arrngmntsvcng-v1-api/master/openapi.yaml	The downstream API Swagger URL can be found in the endpoint of the Downstream API Contract (Swagger Editor), such as this exa https://swagger.mesh.westpac.com.au/?url=https://schema.mesh.westpac.com.au/component/wdp-cap-prodsvcadm-proddealacctadı arrngmntsvcng-v1-api/master/openapi.yaml

downstreamApiPath	Downstream API Contract (Swagger Editor)	/arrangements/summaryView	

Step 1: update dependencies.yaml

Add the downstreamService value found above to the dependencies.yaml file in the corresponding MFE.

```
dependencies:
  apis:
    - name: cap-prodsvcadm-proddealacctadm-instl-arrngmntsvcng-v1
```

Step 2: Run Mesh NPM CLI Command

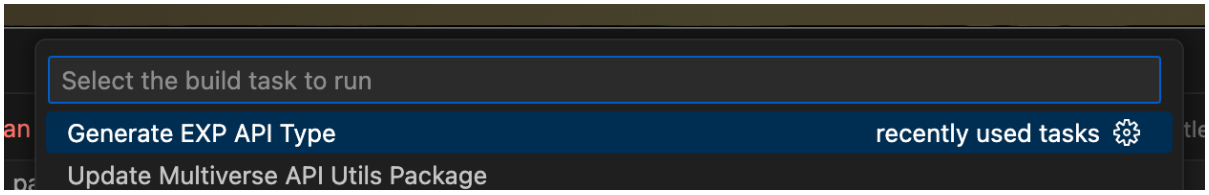
```
npm run policies
npm run routes
npm run api-clients
npm run redux-artifacts
```

> The **"npm run policies"** command will update both the **northEdge.entitlements.json** and **southEdge.entitlements.json** files.
> Please ensure the changes only affect the API you're building.
> And set "allows": ["*"] in the **northEdge.entitlements.json** for the API you're building.

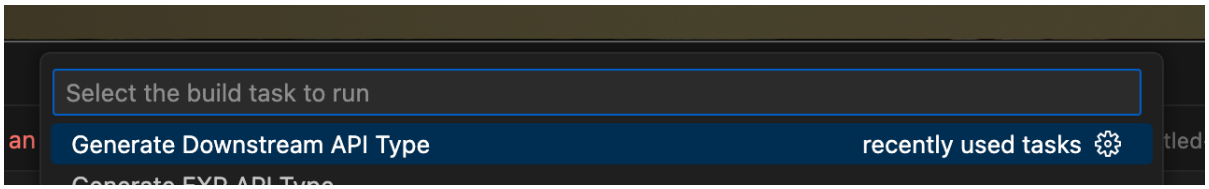
Step 3: Generate EXP API and Downstream API Type

1. Use `Command + Shift + B` to open the VS Code tasks panel.

2. Select the task "**Generate EXP API Type**" to generate or update the type definition of the EXP API.



3. Select the task "**Generate Downstream API Type**" and provide the values for **`"downstreamService"`** and **`"downstreamSwaggerUrl"`** collected earlier to generate or update the type definition of the Downstream API.



Node.js might reject links which requires self-signed certificate, if SELF_SIGNED_CERT_IN_CHAIN error occurs, you can temporarily add the following to the top of the script (generateDownstreamApiType.js) to ignore the error
`process.env.NODE_TLS_REJECT_UNAUTHORIZED = '0';`

Update to API client generation using Mesh Tooling

It looks like there was a change made recently to the Mesh tooling with regards to how the API client generation works (`npm run api-clients`). Now, when generating the API client .js files, the JSDoc comments in these files include type definitions using @typedef. The TypeScript compiler respects JSDoc type annotations from JavaScript files so this is likely to result in failing tests due to typing issues after running `npm run api-clients` for an existing MFE.

This is the change made to the templating used by the API client gen tool:

https://bitbucket.srv.westpac.com.au/projects/WDP/repos/mep-exp-tools-library/diff/packages/api-tools/templates/api-clients-generator/templates/%7B%7BapiName%7D%7DServices.js.mustache?until=ee685f71428707c9fe575599b51518f535fe3fd4

Additionally, it looks like the API client generation process is also generating downstream API types that we need to use with these clients, since they are now typed using JSDoc. Responses from these API clients extend a type called BaseResponse, but this does not look to be included in the type generation, nor is it exported from the mep-exp-tools-library package. You may need to manually define this as a common type if you are regenerating API clients in an MFE for the first time in a while.

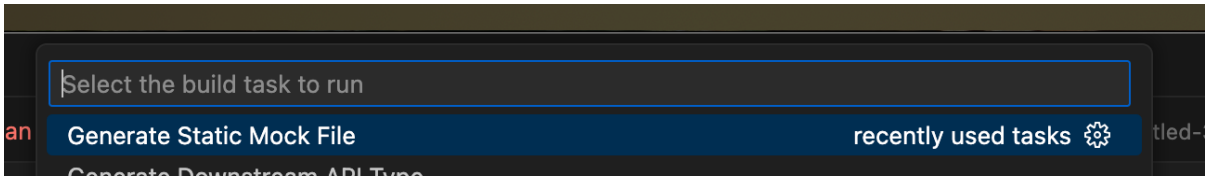
You can review the following PR for the Payment MFE which addresses these changes:

https://bitbucket.srv.westpac.com.au/projects/WDP/repos/galaxy-payment-microfrontend/pull-requests/1418/overview

You can see the BaseResponse in *api/interfaces/types/common.ts*. Check out *api/interfaces/models/downstream/capProdsvcfulflmntPymntprocsngPymntinitV1Services.ts* for an example of API responses that extend BaseResponse.

We probably need to update our process for building EXP APIs with Copilot and VSCode tasks, but this may end up simplifying the downstream API typing that we have to manage.

4. Select the task **"Generate Static Mock File"** and provide the values for **downstreamService**, **downstreamApiPath**, and **downstreamApiHttpMethod** collected earlier to generate a mock file template.



Step 4: Generate `${expApiOperationId}Controller.ts` by GitHub Copilot Prompt

Make sure to replace the value fields in the prompt with the API you're creating.

```
Generate an API controller file using the following template and provided values:
Template:
import { Request, Response, NextFunction } from 'express';
import { normalise } from '@mep-node/framework-response-normaliser-addon';
import { handleControllerError, handleApiService, normaliseData } from '@mesh-tenant-multiverse-ui-common/mv-
apiutils';
import { CONSTANTS } from '../../utils/constants';
import { ${covertToCamelCase(downStreamService)}Services } from '../../generated/api-clients';
import { transformDownstreamResponseToUi, ExpApiResponseData } from '../helpers/${expApiOperationId}Controller.
helper';

const ${expApiOperationId}Controller = async (
  req: Request,
  res: Response,
  next: NextFunction
) => {
  const { logger } = res.locals;
  const {
    httpStatus: { OK },
  } = CONSTANTS;

  const { ${expApiOperationId} } =
    ${covertToCamelCase(downStreamService)}Services;

  try {
    const downstreamResponse = await handleApiService({
      req,
      res,
      apiClient: ${expApiOperationId}
    });

    const expApiResponseData: ExpApiResponseData = transformDownstreamResponseToUi(
      downstreamResponse
    );

    const normalisedData = normaliseData('${schemeKey}', expApiResponseData, normalise)

    res.status(OK).json(normalisedData).end();
  } catch (error) {
    handleControllerError({error, res, logger})
    next(error);
  }
};

module.exports = ${expApiOperationId}Controller;


Values:
- downStreamService: "cap-prodsvcadm-proddealacctadm-instl-arrngmntsvcng-v1"
- expApiOperationId: "getArrangements"
- schemeKey: "arrangements"
```

Step 5: Generate `${expApiOperationId}Controller.helper.ts` by GitHub Copilot Prompt

Make sure to replace the value fields in the prompt with the API you're creating.

```
Generate an API controller helper file using the following template and provided values:

Template:
import { DownstreamServiceResponseData } from '@mesh-tenant-multiverse-ui-common/mv-apiutils';
import { ${capitalizeFirstLetterForCamelCase(expApiOperationId)}Data } from '../../../interfaces/models
/galaxyPaymentExp';

export type ExpApiResponseData = ${capitalizeFirstLetterForCamelCase(expApiOperationId)}Data['entities']
['${schemeKey}'];

const mockExpApiResponseData: ExpApiResponseData = [];

export const transformDownstreamResponseToUi = (
  response: DownstreamServiceResponseData
): ExpApiResponseData => {
  // TODO: Add transformation logic here
  return mockExpApiResponseData;
};

Values:
- downstreamService: "capProdsvcadmProddealacctadmInstlArrngmntsvcngV1"
- expApiOperationId: "getArrangements"
- schemeKey: "arrangements"
- downstreamApiName: "getArrangements"
```

Step 6: Get a sample downstream API response to update the static mock file and update the mockoon-data.json.

Step 7: Fulfill the transformDownstreamResponseToUi function from ${expApiOperationId} Controller.helper.ts based on the data mapping table in the solution design.